

Escuela Técnica N°32 DE N°14

Gral José de San Martín



Informe Final

Trabajo final de PP 6°1

*Edgar Hernandez, Juan Martin Romero, Kevin Arrua, Bruno Bautista y
Lucas Solis*

*juanmartinromero.et32@gmail.com, edgar.hernandezet32@gmail.com,
brunobautistaet32@gmail.com, alexarruaet.32@gmail.com y
lucasa.soliset32@gmail.com*

Docentes de la asignatura

Prof. Gonzalo Consorti

Prof. Luciano Alderete

1. Documentación Inicial del proyecto

Nombre del proyecto: Floraly

El proyecto se desarrolla en el contexto de una florería ubicada en Villa Crespo, dedicada a la venta de ramos florales, plantas, regalos y arreglos personalizados. La Florería tiene que hacer los pedidos de forma presencial, tienen problemas con la gestión del stock, dificultades para los envíos, tienen que realizar una lista de cuidados para cada planta y realizan arreglos personalizados.

Objetivo general:

Actualmente, el negocio cuenta con tienda física y presencia en redes sociales, por eso buscamos desarrollar un sitio web que permita llegar a más clientes y facilitar las compras.

Objetivos Específicos:

- Analizar el proceso actual de gestión.
- Identificar los requisitos Funcionales y no funcionales.
- Una interfaz para poner la ubicación para el envío.
- Diseñar la base de datos del sistema.
- Realizar una interfaz amigable para el usuario y para los dueños.
- Realizar pruebas de funcionamiento.

Alcance:

Una mejor gestión de los productos, una lista de envíos, tener una lista de cuidados para cada planta, una mejor gestión de ingresos y una interfaz con los distintos arreglos personalizados que ofrecen.

Roles de cada integrante:

1. Juan: Scrum Master

Es la líder del proceso y responsable de la calidad documental.

Roles Scrum: Definición formal del Product Owner y Scrum Master.

1° Planeación de los Sprints: Organización de días de trabajo y tableros.

Gestión de Entrega: Responsable de subir todo a la Branch del GIT y asegurar el formato

APA en todo el informe.

2. Alexander: Analista Funcional (Product Owner)

Es el nexa con las necesidades del cliente y el negocio.

Encuestas y Entrevistas: Ejecución con metodología diamante (preguntas abiertas/cerradas).

Descripción de la problemática: Documentar por qué el sistema actual falla y causa lentitud.

Análisis de Requerimientos: Tabla de requerimientos funcionales y no funcionales con sus justificaciones.

Propuesta funcional: Definición de qué hará el sistema para optimizar las ventas.

Diagrama GANTT + Camino Crítico (PERT): Creación del cronograma en ProjectLibre incluyendo recursos y costos.

DER (Diagrama Entidad-Relación): Modelado con normalización 1°, 2° y 3° forma.

Diagrama de caso de uso Simplificado: Representar cómo los actores interactúan con los datos del sistema.

3. Lucas: Diseñador UX/UI y QA (Tester)

Su foco es la experiencia del usuario y verificar que todo funcione.

Bocetos de las vistas / Prototipado Rápido: Creación de las pantallas en Figma o Canvas.

Descripción de la solución (Vista Usuario): Explicar cómo el diseño facilitará la búsqueda exacta por marca y modelo.

Pruebas Funcionales: Realizar el testeo de los módulos una vez desarrollados para asegurar que cumplen los objetivos.

4. Juan: Arquitecto de Software / POO (Backend)

Encargado de la lógica interna y la estructura del código.

Elección de lenguajes: Justificación técnica del stack (ej: PHP o lenguajes de objetos).

Descripción de la solución (Técnica): Explicar la arquitectura (ej: cliente-servidor).

Estimación de entrega (MVP): Definir qué alcance técnico tendrá la primera versión funcional.

5. Bruno: Desarrollador Frontend

Responsable de lo que el usuario ve y toca en la web.

Elección de entorno: Justificar por qué se eligió Web (según pedido del cliente) sobre Mobile o Escritorio.

Implementación de interfaz: Traducir los bocetos de Córdoba a código (HTML, CSS, JS) asegurando que sea responsivo (celular y PC).

6. Edgar: DBA (Administrador de Base de Datos)

Encargado de la integridad y estructura de la información.

Armar la DB SQL: Generación del archivo .sql final con tablas para vehículos, marcas y usuarios.

Existen distintas soluciones digitales para la venta de flores, como plataformas de comercio como Shopify y WooCommerce, sitios web propios de florerías y aplicaciones de delivery. Estos sistemas permiten mostrar productos, gestionar pedidos y coordinar entregas.

Los sistemas actuales permiten comprar sin asistir al local, amplían el alcance del negocio y automatizan pedidos, pero pueden ser complejos, costosos o poco personalizados. En muchos casos presentan limitaciones en la gestión del stock, la organización de envíos y la personalización de arreglos. en nuestra florería, esto se refleja en problemas como pedidos solo presenciales, dificultades con el stock, envíos poco organizados y falta de información sobre el cuidado de las plantas.

Para resolver estos problemas se utilizan aplicaciones web desarrolladas con HTML, CSS y JavaScript,PHP , junto con bases de datos como MySQL o PostgreSQL para gestionar productos, pedidos y clientes. Para la parte de framework tenemos pensado usar React

En general, los sistemas actuales facilitan la venta online, pero no cubren las necesidades específicas de una florería. Por ello, nuestro proyecto busca ofrecer una solución adaptada, mejorando la gestión del stock, los envíos, los arreglos personalizados y la información sobre el cuidado de las plantas, además de permitir realizar pedidos de forma online de manera más simple y eficiente para nuestra florería.

3

DEFINICIÓN DEL PROBLEMA

La florería realiza todos sus procesos de manera presencial, tanto la atención al cliente como la gestión de pedidos y la elaboración de arreglos. No cuenta con un sistema digital que permita organizar la información, por lo que el control del stock se realiza de forma manual. Esto implica que los registros de productos, ventas y disponibilidad pueden ser pocos precisos, dificultando la toma de decisiones y la organización del trabajo diario.

La falta de un sistema digital provoca dificultades en el funcionamiento del negocio. Entre ellas se destacan los errores en los registros relacionados con productos y empleados, la posible pérdida de información importante y un bajo control sobre el stock disponible.

Actores del sistema

- **dueños:** se encargan de la gestión general del negocio, incluyendo el control de stock, la supervisión de pedidos y la toma de decisiones.
- **empleados:** responsable de la elaboración de los arreglos, la atención al cliente y la actualización de información básica.
- **clientes:** realizan pedidos, solicitan información sobre productos y requieren servicios de entrega.

Análisis FODA

Fortalezas:

- Amplia variedad de productos.
- Producto más rentable y demandado(flores/plantas).
- Arreglos personalizados.
- Trabajo con eventos.
- Posibilidad de programar entregas.
- Amplía horario de atención.

Debilidades:

- Problemas de stock.
- Pérdidas.
- No se puede cancelar pagos.
- Lógica limitada.

- Sistemas de pagos y operaciones

Oportunidades:

- Crecimiento mediante el sitio web.
- Llega a más clientes fuera de la zona.
- Fechas especiales.
- Posibilidad de expansión.
- Integración de métodos de pago.

Amenazas:

- Variación estacional.
- Saturación de atención (dudas del cliente).
- Cambios de precio (proveedor).
- Dependencia de los eventos especiales, para grandes ventas.
- Cambiar de tendencias de consumo.

4. SOLUCIÓN PROPUESTA

Enfoque general de la solución

Para resolver los problemas en la gestión manual de la florería, se propuso el desarrollo de un sistema web que permita digitalizar los procesos principales del negocio. Este sistema facilitará la gestión del stock, los pedidos, los envíos y la información de los productos, reemplazando los métodos manuales actuales y reduciendo errores en los registros.

Descripción global del sistema

La solución consiste en una página web accesible desde computadoras o dispositivos móviles con conexión a internet. El sistema permitirá a los clientes visualizar los productos disponibles, realizar pedidos y completar datos de envío, mientras que los administradores podrán gestionar el stock, los arreglos personalizados, los pedidos y la información sobre el cuidado de las plantas. De esta manera, se integrarán en una sola plataforma las funciones principales del negocio, mejorando la organización y la atención al cliente.

Justificación de la elección técnica

Se opta por desarrollar una aplicación web debido a su facilidad de acceso desde distintos dispositivos sin necesidad de instalación, su bajo costo de implementación y su mantenimiento sencillo. Además, este tipo de solución permite escalar el sistema en el futuro, incorporar nuevas funcionalidades y adaptarse a las necesidades del negocio. También resulta adecuada considerando que el local ya dispone de computadoras, lo que facilita su adopción.

Estructura general del proyecto

El sistema estará organizado en distintos módulos que permitirán dividir sus funcionalidades de manera clara. Entre ellos se incluirá un módulo de gestión de productos y stock, encargado de registrar y actualizar la información de flores y plantas; un módulo de pedidos, que permitirá gestionar las compras realizadas por los clientes; un módulo de envíos, donde se administrarán las direcciones y entregas; y un módulo de administración, que facilitará el control general del sistema. Esta estructura permitirá un funcionamiento ordenado, facilitando tanto el desarrollo como el uso del sistema.

4.1. ANÁLISIS DE REQUISITOS

Requisitos funcionales:

ID	Requisito No Funcional (RNF)	Criterio de Aceptación / Meta
RNF-01	Usabilidad (UX/UI)	Interfaz intuitiva; el usuario debe poder completar una compra en menos de 5 pasos.

RNF-02	Diseño Responsivo	Adaptación total a móviles (Mobile First), dado que la mayoría de las compras son por celular.
RNF-03	Disponibilidad (Uptime)	Garantía de funcionamiento del 99.9% anual, crítico en fechas como San Valentín o Día de la Madre.
RNF-04	Seguridad de Datos	Uso de certificados SSL (HTTPS) y cumplimiento de estándares PCI-DSS para pagos.
RNF-05	Rendimiento y Velocidad	Carga de la página principal en menos de 2.5 segundos para evitar el abandono del carrito.
RNF-06	Escalabilidad	Capacidad del servidor para soportar picos de tráfico (10x el tráfico normal) en fechas especiales.
RNF-07	Confiabilidad y Backup	Copias de seguridad diarias de pedidos y clientes; recuperación ante fallos en menos de 1 hora.
RNF-08	Integridad de Datos	Validación de formularios para evitar errores en direcciones de entrega o números de teléfono.
RNF-09	Optimización de Recursos	Compresión automática de imágenes de alta resolución a formatos ligeros (WebP) sin perder calidad visual.

RNF-10	Privacidad (GDPR/LPDP)	Cumplimiento con las leyes locales de protección de datos personales de compradores y terceros.
RNF-11	Mantenibilidad	Código modular y documentado para permitir la integración futura con apps de logística (ej. PedidosYa).
RNF-12	Localización	El sistema debe manejar correctamente la zona horaria local para la programación exacta de envíos.

ID	Requisito Funcional (RF)	Descripción Detallada
RF-01	Catálogo dinámico y categorización	Filtros avanzados por tipo (flores, plantas), ocasión (aniversario, duelo, cumpleaños) y color.
RF-02	Gestión de stock y estacionalidad	Control de disponibilidad según temporada y alertas automáticas de falta de insumos frescos.
RF-03	Personalización de arreglos	Interfaz para elegir cantidad de varas, tipo de envoltorio, color de cinta o base (florero, caja).
RF-04	Motor de búsqueda inteligente	Búsqueda por palabras clave, sentimientos (ej. "amor", "pérdida") o nombres específicos de especies.

RF-05	Gestión de usuarios y perfiles	Registro de clientes con historial de pedidos. Opción de "Compra como Invitado" para reducir fricción.
RF-06	Pasarela de pagos integrada	Procesamiento de tarjetas, transferencias y billeteras virtuales con validación en tiempo real.
RF-07	Logística y Programación	Selección de fecha exacta y franja horaria (mañana/tarde). Bloqueo de fechas por cupo lleno.
RF-08	Panel administrativo (Backoffice)	Gestión total de catálogo, precios, promociones de temporada y visualización de rutas de reparto.
RF-09	Diferenciación Comprador/Destinatario	Formulario doble para capturar datos de quien paga y datos de quien recibe (Nombre, Dirección, Teléfono).
RF-10	Módulo de Dedicatorias	Campo de texto para mensajes personalizados que se imprimirán en la tarjeta de regalo.
RF-11	Venta Sugerida (Up-selling)	Sugerencia automática de productos adicionales (bombones, globos, peluches) antes de finalizar compra.
RF-12	Notificaciones de Estado	Envío de alertas vía Email/WhatsApp cuando el pedido sale del local y cuando es entregado.

RF-13	Gestión de Zonas de Envío	Configuración de costos de envío diferenciados por barrios, códigos postales o radios de distancia.
RF-14	Facturación Electrónica	Generación automática de comprobantes según la normativa vigente (DNI/CUIT opcional en checkout).

4.2. *ESPECIFICACIONES DEL SISTEMA*

El sistema será desarrollado bajo una arquitectura cliente-servidor, donde el cliente accederá a la aplicación mediante un navegador web desde cualquier dispositivo con conexión a internet. El servidor será el encargado de procesar la lógica del sistema, gestionar los pedidos, controlar el stock y administrar la información. Por su parte, la base de datos almacenará de forma estructurada los datos de productos, clientes, pedidos y envíos, permitiendo un acceso rápido y organizado a la información.

Tecnologías empleadas

El sistema utilizará tecnologías web estándar para su desarrollo. En el frontend se emplearán HTML, CSS y JavaScript para la construcción de la interfaz de usuario. En el backend se utilizará PHP para gestionar la lógica del sistema. La base de datos será MySQL, encargada de almacenar toda la información. Además, se utilizará un servidor local como XAMPP durante la etapa de desarrollo y pruebas.

Justificación de cada tecnología

Se eligieron estas tecnologías por ser accesibles y ampliamente utilizadas en el desarrollo web. PHP y MySQL permiten desarrollar aplicaciones dinámicas con un buen rendimiento y facilidad de integración. Por su parte, HTML, CSS y JavaScript garantizan una interfaz compatible con la mayoría de los navegadores, permitiendo una experiencia de usuario simple e intuitiva. El uso de XAMPP facilita el desarrollo local al integrar servidor web y base de datos en un solo entorno.

Requerimientos mínimos para funcionamiento

El sistema no requiere hardware de alta gama para su funcionamiento. Se recomienda un equipo con procesador Dual Core, al menos 4 GB de memoria RAM y 1 GB de espacio disponible en disco. En cuanto al software, será necesario contar con un sistema operativo como Windows 10 o superior, un navegador web actualizado (como Chrome o Firefox) y un servidor web con soporte para PHP y MySQL, como el proporcionado por XAMPP.

Estructura de módulos

El sistema estará organizado en distintos módulos que permitirán dividir sus funcionalidades principales. Incluirá un módulo de gestión de productos y stock para registrar flores, plantas y arreglos; un módulo de pedidos para administrar las compras realizadas por los clientes; un módulo de envíos para gestionar direcciones y entregas; un módulo de clientes para almacenar la información de los usuarios; y un módulo de administración que permitirá el control general del sistema. Esta estructura modular facilitará tanto el desarrollo como el mantenimiento del sistema.

4.3. ANALISIS DEL SISTEMA

El sistema Floraly se implementa exclusivamente como un sitio web cliente-servidor, accesible desde cualquier navegador sin necesidad de instalación. Esta decisión se basa en que la mayoría de los

clientes acceden desde redes sociales y dispositivos móviles, y en la necesidad del local de centralizar pedidos y stock en una sola plataforma.

4.3.1 Diagramas y Bocetos

Para representar gráficamente el funcionamiento del sistema se elaboraron los siguientes diagramas y bocetos: https://github.com/Practicas-Profesionalizantes/2026-6-1-Grupo_2--e-Commerce-Floreria--/tree/main/diagramas-y-bocetos

4.3.2 Diseño de Base de Datos

Modelo Entidad-Relación (MER)

El sistema se basa en un modelo entidad-relación normalizado que permite gestionar de forma eficiente usuarios, productos, pedidos, envíos y stock.

Entidades principales

- Usuario
- Rol
- Dirección
- Producto
- Categoría
- Tipo de Producto
- Tamaño
- Pedido
- Detalle de Pedido
- Pago
- Factura

- Envío
- Stock

Relaciones principales

- Un Usuario puede realizar muchos Pedidos (1:N)
- Un Pedido contiene muchos Productos a través de la tabla Detalle de Pedido (N:M)
- Un Pedido tiene un solo Envío (1:1)
- Un Pedido genera un Pago y una Factura (1:1)
- Un Producto pertenece a una Categoría y a un Tipo de Producto (N:1)
- Un Usuario puede tener múltiples Direcciones (1:N)

Tablas del sistema

Usuarios

- id_usuario (PK)
- nombre
- apellido
- email
- contraseña_hash
- dni
- telefono
- fecha_registro
- id_rol (FK)
- activo

Roles

- id_rol (PK)
- nombre_rol (Cliente / Empleado / Administrador)

Direcciones

- id_direccion (PK)
- id_usuario (FK)
- calle
- numero
- piso
- depto
- localidad
- codigo_postal
- referencia

Productos

- id_producto (PK)
- nombre
- descripcion
- precio
- id_categoria (FK)
- id_tipo_producto (FK)
- id_tamano (FK)
- stock_actual
- stock_minimo
- imagen
- activo

Categorías, Tipos de Producto, Tamaños

(Tablas catálogo)

Pedidos

- id_pedido (PK)
- id_usuario (FK)
- fecha_creacion
- estado (Pendiente / Confirmado / En preparación / Enviado / Entregado / Cancelado)
- total
- observaciones

Pedido_detalle

- id_detalle (PK)
- id_pedido (FK)
- id_producto (FK)
- cantidad
- precio_unitario

Pagos / Envíos / Facturas

(con sus campos correspondientes)

4.3.3 Arquitectura del Sistema

Arquitectura Cliente-Servidor

El sistema está estructurado en tres capas claramente diferenciadas:

1. Capa de Presentación (Frontend)

- Interfaz visible para el usuario.
- Tecnologías: HTML5, CSS3, JavaScript, Bootstrap 5.
- Enfocada en usabilidad, diseño responsive (Mobile First) y experiencia de usuario intuitiva.

2. Capa de Lógica de Negocio (Backend)

- Contiene toda la inteligencia del sistema.
- Tecnologías: PHP 8.
- Responsable de validar datos, gestionar reglas de negocio, procesar pedidos, controlar stock y manejar pagos.

3. Capa de Datos

- Motor: MySQL
- Almacena toda la información de forma segura y estructurada.
- Incluye procedimientos almacenados y triggers para mantener la integridad de los datos (ej. control de stock).

Justificación de la arquitectura por capas

- Facilita el mantenimiento y escalabilidad.
- Permite trabajar en paralelo (frontend y backend).
- Mejora la seguridad al separar la lógica de la presentación.
- Simplifica futuras integraciones (ej. aplicación móvil, sistema de delivery externo).

4.3.4 Tecnologías Utilizadas

Capa	Tecnología	Justificación
Frontend	HTML5, CSS3, JavaScript, Bootstrap 5	Diseño minimalista, responsive y rápido de implementar
Backend	PHP 8	Fácil integración con MySQL, adecuado para proyectos educativos y de mediana complejidad
Base de Datos	MySQL	Robusta, gratuita y ampliamente compatible

Librerías	SweetAlert2, Axios	Mejora de experiencia de usuario y comunicación asíncrona
Entorno Desarrollo	XAMPP	Integración completa (Apache + MySQL + PHP)

5

El Proyecto tuvo las siguientes etapas:

- 1. Relevamiento de información.**
- 2. Análisis del problema.**
- 3. Definición de requisitos.**
- 4. Diseño del sistema.**
- 5. Diseño de base de datos.**
- 6. Desarrollo del sistema.**
- 7. Pruebas funcionales.**
- 8. Corrección de errores.**
- 9. Documentación final.**

Descripción:

1. Relevamiento de información

Se realizó una entrevista en profundidad con los dueños de la florería ubicada en Villa Crespo para comprender el funcionamiento actual del negocio, los procesos manuales de atención al cliente, gestión de stock, elaboración de arreglos personalizados y organización de envíos.

2. Análisis del problema

Se identificaron y documentaron las principales dificultades del negocio: control manual de stock con riesgo de faltantes y pérdidas, pedidos exclusivamente presenciales, dificultades en la organización de envíos, falta de información digital sobre cuidados de las plantas y limitaciones en la atención durante fechas de alta demanda.

3. Definición de requisitos

Se elaboraron los requisitos funcionales y no funcionales del sistema, priorizando aquellas funcionalidades que resuelven los problemas detectados (catálogo dinámico, personalización de arreglos, gestión de stock, proceso de compra con envío y panel administrativo). Se utilizó la metodología del doble diamante para la realización de encuestas y entrevistas.

4. Diseño del sistema

Se definieron la arquitectura general cliente-servidor en tres capas, se elaboraron diagramas UML (casos de uso, secuencia y clases), bocetos y prototipos de interfaz de usuario en Figma, y la estructura modular del sistema (catálogo público, gestión de pedidos, envíos y administración).

5. Diseño de base de datos

Se modeló el Diagrama Entidad-Relación (MER), se aplicó normalización hasta la 3ª forma normal y se definieron las tablas principales (Usuarios, Roles, Productos, Categorías, Pedidos, Detalle de Pedido, Envíos, Pagos, Facturas, Stock, etc.), incluyendo claves primarias, foráneas y restricciones de integridad de datos.

6. Desarrollo del sistema

Se implementarán los módulos principales del sistema utilizando las tecnologías seleccionadas:

Frontend: HTML5, CSS3, JavaScript y Bootstrap 5 para lograr una interfaz moderna, intuitiva y completamente responsive (Mobile First).

Backend: PHP 8 para gestionar la lógica de negocio, validaciones, control de sesiones y procesamiento de pedidos.

Base de datos: MySQL para el almacenamiento seguro y estructurado de la información.

El desarrollo se realizará trabajando en paralelo entre el equipo de frontend y backend, integrando progresivamente las funcionalidades a través de endpoints internos.

7. Pruebas funcionales

Se realizarán pruebas unitarias, de integración y de sistema completo, incluyendo el flujo de compra desde el catálogo hasta la confirmación del pedido, gestión de stock, login de usuarios y administrador, y compatibilidad en diferentes dispositivos y navegadores. Lucas Solís, responsable de QA, coordinará estas pruebas para asegurar que todas las funcionalidades cumplan con los requisitos definidos.

8. Corrección de errores y ajustes

Una vez finalizadas las pruebas, se identificarán y resolverán los bugs detectados, se optimizará el rendimiento (tiempos de carga de las páginas), se mejorará la usabilidad de la interfaz según el feedback recibido y se realizarán los ajustes necesarios para garantizar una experiencia de usuario óptima.

9. Documentación final y cierre del proyecto

Se redactará el presente informe técnico completo, se elaborará el manual de usuario para los administradores y clientes, se organizará y documentará el repositorio en GitHub, y se preparará la presentación final del proyecto para su defensa ante los docentes.

Camino Crítico

ID	Actividad	Predecesora(s)	Duración (semanas)
A	Relevamiento de información	—	2
B	Definir estructura del sitio y funciones	A	2
C	Diseño de base de datos	B	3
D	Diseño de prototipos de pantallas	B	3
E	Redacción de descripciones de productos	B	1
F	Diseño de la API (endpoints, rutas, pruebas)	C	4
G	Desarrollo de la API (login, productos, stock, pagos, DNI)	F	8
H	Pruebas de la API (Postman)	G	2
I	Desarrollo del frontend (diseño visual)	D	6
J	Conexión del frontend con la API	H, I	4
K	Carga de productos al sistema	E, G	5
L	Integración redes sociales y contacto	I	2
M	Pruebas generales del sistema	J, K	3
N	Correcciones finales	M	2
O	Publicación del sitio	N	1

El plan está organizado de manera lógica y profesional, respetando las etapas clásicas del desarrollo de software:

1. Análisis y diseño: relevamiento, definición de estructura, base de datos y prototipos.
2. Desarrollo del backend (API): diseño, programación y pruebas.
3. Desarrollo del frontend: diseño visual y luego conexión con la API.
4. Integración de contenidos: carga de productos y redes sociales.
5. Pruebas, correcciones y publicación.

La ruta crítica del proyecto (la que determina el tiempo total) pasa principalmente por estas tareas:

$A \rightarrow B \rightarrow C \rightarrow F \rightarrow G \rightarrow H \rightarrow J \rightarrow M \rightarrow N \rightarrow O$

Esto demuestra que el núcleo del proyecto es la API y su integración con el frontend. Si alguna de estas tareas se retrasa, se retrasa todo el proyecto.

Mientras tanto, otras actividades como la redacción de productos (E) o la integración de redes sociales (L) pueden hacerse en paralelo y no afectan el tiempo final.

Duración total del proyecto: 31 semanas

En conclusión, el cronograma está muy bien planificado para un proyecto anual, permite trabajar en paralelo en varias partes del sistema, reduce riesgos y asegura que antes de publicar el sitio se hayan realizado todas las pruebas y correcciones necesarias para que funcione correctamente.

Recursos utilizados

- Visual Studio Code
- Draw.io

- MySQL Workbench
- ProjectLibre
- Paint
- React
- Node.js
- Computadoras de Desarrollo